Infrastruktur Big Data: Optimasi & Tuning Pipeline

Materi Kuliah Mendalam – Pertemuan 12

Fakultas Ilmu Komputer

Capaian Pembelajaran Hari Ini

Setelah menyelesaikan pertemuan ke-12 ini, mahasiswa diharapkan wajib menguasai:

- Evaluasi kelemahan format file teks (CSV/JSON) pada klaster produksi.
- Arsitektur internal penyimpanan kolom (Apache Parquet).
- Mekanisme Caching dan Persistence untuk efisiensi memori DAG.
- Teknik manipulasi partisi data melalui Repartition dan Coalesce.
- Strategi deteksi dan mitigasi masalah ketimpangan data (Data Skewness).

Kilas Balik: Perjalanan Pertemuan 1 - 11

Fase Dasar & Praktikal

Pertemuan 1–7: Kita mendalami teori MapReduce, HDFS, arsitektur Master-Worker, ekosistem YARN, dan dasar komputasi memori Spark RDD.

Pertemuan 8–11: Anda telah melewati fase instalasi di Windows, konfigurasi YARN classpath, serta pemecahan masalah Py4J Connection Error.

Status Mahasiswa Saat Ini:

Anda sudah bisa membuat program Spark yang "berjalan dengan sukses". Mulai hari ini, fokus kita bergeser: **Bagaimana membuat program tersebut berjalan dengan cepat dan efisien.**

Paradoks Big Data: Skala vs Efisiensi Biaya

Menambah jumlah node komputer (scaling out) bukan satu-satunya jawaban saat sistem mengalami kelambatan.

> **Prinsip Utama Arsitek Big Data:**
>
> "Kode yang ditulis dengan buruk pada klaster berkapasitas 100 node akan berjalan lebih lambat dan membuang anggaran cloud dibandingkan kode yang telah dioptimalkan dengan baik pada klaster berkapasitas 10 node."

Optimasi di level perangkat lunak dan format penyimpanan adalah kunci efisiensi biaya perusahaan.

Peta Perjalanan Hari Ini (5 Bab Utama)

1. Limitasi File Teks

Mengapa CSV/JSON dilarang di lingkungan produksi Big Data.

2. Apache Parquet

Bedah anatomi dan teknologi penyimpanan berbasis kolom.

3. Spark Caching

Teknik mengunci data di RAM untuk menghindari re-komputasi.

4. Tuning Partisi Data

Strategi penyeimbangan data lewat Repartition & Coalesce.

5. Data Skewness

Identifikasi dan penanganan ketidakseimbangan beban kerja worker.

BAB 1: Mengapa Format File Tradisional Gagal di Produksi?

Analisis Kelemahan Sistem Penyimpanan Berbasis Teks

Anatomi CSV: Struktur Fisik dan Limitasi

CSV (Comma-Separated Values) adalah format penyimpanan berbasis baris (*row-based text format*).

- **Penyimpanan Data Berurutan:** Data disimpan per baris secara horizontal di dalam disk.
- **Pemisah Karakter:** Menggunakan karakter pembatas (koma, titik koma, tab) untuk memisahkan setiap nilai kolom.
- **Ketiadaan Metadata Struktural:** File CSV tidak membawa informasi tipe data bawaan. Kolom angka, tanggal, dan teks semuanya ditulis sebagai karakter mentah.

Hambatan Parsing Teks dalam Skala Terabyte

Saat Anda membaca file CSV berukuran 1 TB, Spark harus melakukan konversi tipe data secara masif.

Beban CPU Tersembunyi (Overhead Parsing):

Setiap baris teks harus dibaca sebagai senarai karakter, lalu dipotong berdasarkan koma, kemudian diubah secara eksplisit oleh CPU menjadi objek Java tipe Integer, Float, atau Timestamp. Proses penafsiran teks ini menghabiskan banyak siklus kerja CPU.

Masalah Fragmentasi Blok HDFS pada File CSV

Bagaimana HDFS memotong file teks biasa?

- HDFS memotong file berdasarkan ukuran blok default (misal 128 MB), tanpa peduli letak baris teks kalimat.
- **Record Splitting Problem:** Baris teks di akhir batas 128 MB sering kali terpotong di tengah kalimat.
- **Dampak Performa:** Worker Node yang memproses Blok A terpaksa melakukan koneksi jaringan ke Worker Node Blok B hanya untuk membaca sisa potongan kata di baris tersebut. Ini memicu *Network I/O* yang tidak perlu.

I/O Bottleneck: Beban Pemindahan Kolom Berlebih

Bayangkan sebuah tabel log server memiliki 100 kolom data, namun analis Anda hanya membutuhkan kolom `IP_Address` dan `Status_Code`.

```
SELECT IP_Address, Status_Code FROM web_log_csv WHERE status = 500
```

Pada file CSV, sistem penyimpanan **wajib membaca seluruh 100 kolom** dari harddisk ke memori karena struktur datanya bergandengan per baris. 98 kolom sisanya dibaca lalu langsung dibuang, menyebabkan pemborosan bandwidth disk secara masif.

Skenario Masalah Nyata: Pemindaian Skala Besar

Kasus CSV (1 TB Data Log)

- Total Kolom: 50
- Kolom Di-query: 2
- Data Dibaca dari Disk: **1 TB**
- Waktu Pemrosesan: ~15 Menit

Kasus Parquet (Dataset Sama)

- Total Kolom: 50
- Kolom Di-query: 2
- Data Dibaca dari Disk: **~25 GB**
- Waktu Pemrosesan: ~20 Detik

Mengapa RDBMS Bukan Solusi untuk Skala Ini?

Muncul pertanyaan mahasiswa: "Mengapa kita tidak pakai database MySQL atau PostgreSQL saja?"

- RDBMS tradisional dirancang untuk transaksi OLTP (Online Transaction Processing) berskala kecil dan cepat, bukan analisis data skala besar (OLAP).
- RDBMS menggunakan skema penguncian baris (row locking) dan memiliki batasan penyimpanan fisik pada satu server tunggal (*single point of failure*).
- Sistem klaster Big Data mendistribusikan data mentah tanpa sistem indeks b-tree tradisional yang memakan memori besar.

BAB 2: Bedah Arsitektur Apache Parquet

Teknologi dan Mekanisme Internal Penyimpanan Kolom

Apa itu Columnar Storage? Konsep Dasar

Format penyimpanan kolom membalikkan cara data ditulis ke dalam harddisk.

Row-Based (CSV):

Baris1_Kol1, Baris1_Kol2, Baris1_Kol3
Baris2_Kol1, Baris2_Kol2, Baris2_Kol3

Columnar-Based (Parquet):

Baris1_Kol1, Baris2_Kol1 | Baris1_Kol2, Baris2_Kol2 |
Baris1_Kol3, Baris2_Kol3

Dengan cara ini, nilai-nilai dalam satu kolom yang sama diletakkan berdampingan secara fisik di dalam blok penyimpanan disk.

Anatomi Internal Parquet: Row Groups

File Parquet tidak menyimpan seluruh kolom dari atas sampai bawah secara masif langsung jadi satu kesatuan, melainkan membaginya menjadi unit horizontal bernama **Row Groups**.

- Setiap *Row Group* biasanya berisi kumpulan data dari 100.000 hingga 1.000.000 baris.
- Di dalam satu Row Group, barulah data dipotong per kolom secara vertikal.
- Tujuan: Menjaga agar chunk data tetap muat di dalam memori buffer RAM worker saat proses penulisan dan pembacaan berlangsung.

Anatomi Internal Parquet: Column Chunks & Pages

Mari kita bedah lebih dalam struktur di dalam satu Row Group:

- **Column Chunk:** Blok data fisik yang menyimpan nilai khusus untuk satu kolom tertentu di dalam satu Row Group.
- **Data Page:** Unit terkecil di dalam Column Chunk tempat algoritma kompresi data dijalankan. Ukuran satu Page biasanya berkisar 1 MB.
- Setiap Page memiliki header tersendiri yang menyimpan informasi metadata enkripsi dan cek validitas data (checksum).

Mekanisme Kunci: Projection Pushdown

Bagaimana Spark memanfaatkan Parquet untuk menghemat pembacaan data disk secara drastis?

Definisi Projection Pushdown:

Kemampuan mesin query untuk mengidentifikasi kolom apa saja yang diminta dalam sintaks SQL, lalu menginstruksikan modul pembaca file agar **hanya memuat byte lokasi Column Chunk dari kolom tersebut saja**, mengabaikan kolom lainnya secara total di level penyimpanan fisik disk.

Mekanisme Kunci: Predicate Pushdown

Parquet menyimpan data statistik (Min dan Max value) dari setiap kolom di setiap ujung file.

Cara Kerja Predicate Pushdown (Filter Pushdown):

Jika query Anda berisi perintah: `WHERE age > 30`, Spark akan memeriksa catatan statistik internal Row Group 1. Jika tercatat nilai `Max_Age = 28` pada Row Group tersebut, Spark akan langsung **melewati (skip) Row Group 1 secara keseluruhan tanpa membacanya sama sekali**.

Kompresi Pintar: Dictionary Encoding

Karena satu kolom berisi data bertipe sejenis, Parquet dapat melakukan kompresi tingkat tinggi menggunakan **Dictionary Encoding**.

Misal kolom `Country` berisi kata berulang: "Indonesia", "Malaysia", "Indonesia".

Kamus Internal yang Dibuat Parquet:
0 -> Indonesia, 1 -> Malaysia

Data fisik yang disimpan di disk berubah menjadi deretan angka biner pendek: `0, 1, 0`, menghemat ruang penyimpanan teks hingga berkali-kali lipat.

Kompresi Pintar: Run-Length Encoding (RLE)

Metode kompresi kedua yang sering dikombinasikan di dalam Parquet adalah Run-Length Encoding (RLE).

Misalkan data log server Anda berurutan memiliki Status Code sukses yang seragam:

```
200, 200, 200, 200, 200, 200, 200, 200
```

RLE akan mengompresi deretan panjang tersebut dan menyimpannya dalam bentuk format ringkas pasangan frekuensi:

```
5 * 200 (Artinya: angka 200 muncul berurutan sebanyak 5 kali)
```

Pilihan Algoritma Kompresi: Snappy vs GZip

Saat data disimpan ke format Parquet, kita bisa membungkusnya dengan algoritma kompresi sekunder:

- **Snappy (Default):** Dirancang bukan untuk mengecilkan ukuran seminimal mungkin, melainkan untuk kecepatan proses kompresi dan dekompresi bit data yang sangat tinggi dengan beban CPU rendah. *Sangat disarankan untuk pipeline Big Data harian.*
- **GZip / ZSTD:** Menghasilkan file dengan ukuran jauh lebih kecil dari Snappy, namun memakan daya komputasi CPU yang besar saat proses bongkar-pasang file dilakukan. *Cocok untuk data arsip historis jangka panjang.*

Validasi Tipe Data & Schema Preservation

Berbeda dengan file teks biasa yang rentan rusak akibat perubahan posisi baris kolom, Parquet bersifat **Self-Describing**.

- Struktur nama kolom, tipe data presisi (seperti `Decimal(10,2)`, `Timestamp`), dan informasi status nilai kosong (nullability) dikunci di dalam footer file Parquet.
- Saat Spark membaca file Parquet, Spark langsung mengetahui cetak biru data tersebut tanpa perlu menerka-nerka tipe datanya lagi, meminimalisir bug data corupt di tengah jalan.

BAB 3: Implementasi Hands-on Parquet di PySpark

Panduan Sintaksis Penulisan, Pembacaan, dan Tuning Parameter

Kode: Menulis DataFrame ke Parquet

Berikut adalah contoh sintaks dasar untuk melakukan ekspor DataFrame hasil analisis Anda ke dalam format Parquet:

```
# Membaca data log mentah dari CSV terlebih dahulu
df_mentah = spark.read.csv("D:/data/web_log.csv", header=True)

# Menyimpan DataFrame ke folder target format Parquet
df_mentah.write.parquet("D:/data/output_parquet")
```

Secara default, Spark akan otomatis menerapkan kompresi jenis snappy tanpa perlu deklarasi manual.

Kode: Tuning Parameter Menulis Tingkat Lanjut

Dalam kondisi riil di lingkungan kerja, Anda wajib menggunakan opsi pengaman penulisan file seperti berikut:

```
df_mentah.write \
  .mode("overwrite") \
  .option("compression", "snappy") \
  .parquet("D:/data/output_parquet_tuned")
```

- `mode("overwrite")`: Otomatis menghapus isi folder lama jika sudah ada hasil uji coba sebelumnya, menghindari program crash akibat isu penulisan jalur file baru.

Kode: Menulis dengan Teknik PartitionBy()

Kita bisa menginstruksikan Spark untuk membuat struktur sub-folder fisik di harddisk berdasarkan nilai kolom tertentu:

```
# Membagi penyimpanan file berdasarkan kode status respon server
df_mentah.write \
  .mode("overwrite") \
  .partitionBy("Status_Code") \
  .parquet("D:/data/log_terpartisi")
```

Hasil di dalam Explorer Anda akan membentuk struktur folder rapi: `/log_terpartisi/Status_Code=200/` dan `/log_terpartisi/Status_Code=500/`.

Kode: Membaca File Parquet kembali ke Spark

Membaca Parquet jauh lebih hemat baris instruksi koding karena kita tidak perlu lagi menulis deklarasi parameter pendukung:

```
# Membaca data dari direktori parquet
df_parquet = spark.read.parquet("D:/data/output_parquet_tuned")

# Spark langsung paham tipe kolom masing-masing
df_parquet.printSchema()
```

Keuntungan Kecepatan: Spark tidak perlu memicu Job internal pemindaian baris data awal (*InferSchema*) seperti saat membaca file CSV.

Eksplorasi: Fitur Merge Schema

Bagaimana jika struktur kolom data bisnis Anda berubah seiring waktu berjalan (Schema Evolution)?

```
# Mengizinkan Spark menggabungkan skema file lama dan baru yang berbeda kolom
df_merged = spark.read \
  .option("mergeSchema", "true") \
  .parquet("D:/data/log_terpartisi")
```

Jika file batch minggu lalu memiliki 5 kolom dan batch minggu ini bertambah menjadi 6 kolom, Spark akan otomatis mengisi nilai `Null` pada baris log lama yang tidak memiliki kolom baru tersebut.

BAB 4: Memahami Spark Engine & Biaya Shuffle

Konsep Dasar Proses Pemindahan Data Antar Node Komputasi

Apa itu Spark Shuffle? Musuh Utama Data Engineer

Definisi Operasi Shuffle:

Proses pemindahan, pengelompokan, dan pendistribusian ulang data dari berbagai Worker Node lama ke Worker Node baru di seluruh jaringan lokal klaster agar records dengan kunci (*key*) yang sama berkumpul di satu tempat.

Operasi ini dipicu oleh fungsi agregasi seperti `groupBy()`, `join()`, dan `distinct()`.

Siklus Hidup Query: Directed Acyclic Graph (DAG)

Sebelum menjalankan tugas, Spark membuat peta rencana eksekusi logis bernama DAG.

- DAG membagi rangkaian logika program Anda menjadi potongan-potongan besar bernama **Stages**.
- Batas pemisah (*boundary*) antar Stage ditentukan secara mutlak oleh keberadaan operasi Shuffle di dalam urutan kode program Anda.

Transformasi: Narrow vs Wide Transformation

Narrow (Tanpa Shuffle):

Setiap partisi data input diproses mandiri oleh satu task tanpa perlu melirik data di node lain.

Contoh: `.filter()`, `.map()`, `.select()`

Wide (Memicu Shuffle):

Membutuhkan data dari partisi lain untuk menyelesaikan perhitungan hasil.

Contoh: `.groupBy()`, `.join()`

Biaya Fisik Shuffle: Disk, Network, & CPU

Mengapa kita harus sebisa mungkin meminimalisir terjadinya operasi Wide Transformation?

Saat Shuffle terjadi, tahapan proses fisik berikut dijalankan otomatis di latar belakang:

1. **Shuffle Write:** Node pengirim menulis hasil komputasi sementara mereka ke dalam harddisk lokal masing-masing.
2. **Network Fetching:** Node penerima menarik file sementara tersebut melewati kabel jaringan klaster.
3. **Shuffle Read:** Node penerima membaca data tersebut ke memori RAM mereka untuk dihitung ulang.

Fenomena Disk Spill: Ketika RAM Tidak Lagi Cukup

Bahaya Utama Disk Spill:

Jika volume perpindahan data hasil Shuffle terlalu masif melebihi kapasitas alokasi RAM Worker Node, Spark terpaksa memindahkan sebagian data antrean memori tersebut ke dalam harddisk lokal (*Spill to Disk*). Proses ini memperlambat eksekusi program Anda hingga ratusan kali lipat.

BAB 5: Penguasaan Caching & Persistence

Strategi Penguncian Objek Data pada Lapisan Memori Kerja Spark

Memahami Kembali Prinsip Lazy Evaluation

Spark tidak akan mengeksekusi transformasi data Anda sampai muncul sebuah perintah **Action**.

```
df1 = spark.read.parquet("log.parquet")
df2 = df1.filter(col("status") == 500) # Belum dihitung!

df2.count() # Action 1: File dibaca, filter dijalankan, dihitung
df2.show()  # Action 2: File DIBACA ULANG dari awal!
```

Ketiadaan strategi optimasi memori membuat Spark melakukan pemindaian ulang file disk mentah secara sia-sia.

Mengapa Kita Butuh Caching? Skenario Re-Komputasi

Dengan menerapkan Caching, kita menghentikan kebiasaan Spark membaca ulang data dari titik nol.

> Saat fungsi `.cache()` disisipkan, hasil komputasi dari DataFrame tersebut akan langsung dikunci dan disimpan menetap di dalam RAM klaster setelah Action pertama kali dijalankan. Action kedua dan seterusnya tidak akan lagi menyentuh harddisk, melainkan langsung mengambil data instan dari RAM.

Sintaks: Cara Penggunaan .cache() yang Benar

Perhatikan penempatan fungsi cache pada contoh kasus berikut:

```
# Menyimpan dataframe hasil filter bersih ke memori
df_fokus = df_mentah.filter(col("tahun") == 2026).cache()

# Evaluasi Action pertama memicu proses penguncian memori
total_data = df_fokus.count()

# Evaluasi Action kedua berjalan secepat kilat
df_fokus.groupBy("kategori").sum().show()
```

Mengenal Tingkat Penyimpanan (Storage Levels)

Spark menyediakan berbagai variasi level penyimpanan data cadangan melalui pustaka `StorageLevel`:

- `MEMORY_ONLY`: Data disimpan sebagai objek Java murni un-serialized di RAM. Sangat cepat, namun boros ruang memori.
- `MEMORY_AND_DISK`: Jika RAM penuh, sisa data diturunkan ke disk agar program tidak mengalami error OutOfMemory.
- `MEMORY_ONLY_SER`: Data diubah menjadi deretan byte padat (serialized). Lebih hemat RAM, namun membutuhkan sedikit performa CPU untuk membongkarnya kembali.

Perbedaan Krusial: .cache() vs .persist()

Metode Pemanggilan	Level Penyimpanan Default	Fleksibilitas Kustomisasi
df.cache()	`MEMORY_AND_DISK`	Kaku, tidak bisa diganti konfigurasinya.
df.persist()	`MEMORY_AND_DISK` (Jika kosong)	Bebas memilih parameter StorageLevel lain.

Contoh sintaks kustomisasi persistensi memori:

```
from pyspark.storagelevel import StorageLevel
df.persist(StorageLevel.MEMORY_ONLY_SER)
```

Manajemen Memori: Strategi Eviksi LRU

Apakah data yang di-cache akan selamanya berada di RAM?

- Jawabannya: **Tidak.** Spark menggunakan algoritma manajemen memori **LRU (Least Recently Used)**.
- Jika aplikasi Spark Anda membutuhkan RAM tambahan untuk menjalankan operasi komputasi baru yang lebih berat, Spark akan otomatis menghapus blok data cache yang paling jarang diakses oleh program Anda untuk mengosongkan ruang memori.

Siklus Hidup Cache: Kapan Harus .unpersist()

Membiarkan cache menumpuk di memori tanpa dibersihkan adalah kebiasaan buruk Data Engineer pemula (*Memory Leak*).

```
# Alur kerja pembersihan memori kerja
df_analisis = df_input.groupBy("id").count().cache()
df_analisis.write.parquet("D:/report")

# Lepas kunci data dari RAM sesegera mungkin
df_analisis.unpersist()
```

Langkah penutupan ini menjamin alokasi RAM kembali bersih untuk digunakan oleh baris program selanjutnya.

BAB 6: Tuning Partisi Data (Partitioning)

Strategi Pengaturan Batas Paralelisme dan Penataan Berkas Output

Apa itu Partisi di Spark? Konsep Dasar

Partisi adalah pembagian fisik data menjadi potongan-potongan kecil agar bisa diproses secara paralel oleh Core CPU yang berbeda.

- 1 Partisi Data akan dikerjakan secara eksklusif oleh 1 Spark Task pada 1 Core CPU.
- Jika DataFrame Anda hanya memiliki 1 partisi, maka berapapun jumlah komputer di klaster Anda, **hanya 1 Core CPU yang bekerja**, sementara sisanya menganggur.

Aturan Emas: Menentukan Ukuran Partisi Ideal

Berapa ukuran file ideal untuk satu partisi data di dalam industri?

The 128 MB Rule:

Ukuran partisi terbaik berkisar antara **100 MB hingga 128 MB** per partisi. Jika partisi terlalu kecil (misal berukuran KB), klaster akan kelelahan mengelola jutaan tugas kecil (Overhead Task Scheduling). Jika terlalu besar (misal gigabyte), worker akan kehabisan RAM.

Mengubah Partisi: Mekanisme .repartition()

Fungsi `.repartition(n)` digunakan untuk menaikkan atau menyeimbangkan kembali jumlah partisi data secara merata.

```
# Menaikkan partisi dari 2 menjadi 20 agar proses kerja paralel maksimal
df_ramai = df_kecil.repartition(20)
```

- **Karakteristik Utama:** Melakukan proses **Full Shuffle** di latar belakang.
- Data diacak total melewati kabel jaringan klaster untuk menjamin pembagian porsi data di partisi baru sama rata (*Uniform Distribution*).

Mengubah Partisi: Mekanisme .coalesce()

Fungsi `.coalesce(n)` digunakan khusus untuk **menurunkan** jumlah partisi data menjadi lebih sedikit.

```
# Menggabungkan 200 partisi hasil analisa menjadi 1 partisi sebelum di-export
df_final = df_banyak.coalesce(1)
```

- **Karakteristik Utama:** Tidak memicu terjadinya Full Shuffle besar-besaran.
- Spark hanya menggabungkan partisi-partisi lokal yang berada di dalam satu mesin worker yang sama, sehingga menghemat bandwidth jaringan secara dramatis.

Bahaya Laten: Fenomena Data Skewness

Definisi Data Skewness (Ketimpangan Data):

Kondisi abnormal di mana distribusi data tidak merata, misalnya akibat fungsi GroupBy pada kolom nama negara, di mana data berlabel "Indonesia" berjumlah 90% sedangkan negara lain hanya 10%.

Akibatnya: 1 Worker Node akan bekerja keras berjam-jam menyelesaikan porsi 90% data tersebut (*Straggler Task*), sementara worker node lainnya sudah selesai mendahului dan duduk diam menganggur.

Lab Mandiri: Skenario Kasus Log 10 Miliar Baris

Tugas Pengayaan Mahasiswa untuk minggu ini:

1. Ambil berkas log web server dari tugas praktikum ke-11 kemarin.
2. Ubah format penyimpanan ekspor data akhir yang tadinya berupa berkas CSV biasa menjadi format **Apache Parquet**.
3. Terapkan fungsi penyeimbang partisi `.coalesce(1)` tepat sebelum baris kode simpan file dijalankan, jalankan program, lalu amati perbedaan struktur file output di dalam folder penyimpanan lokal Anda.

Sesi Tanya Jawab & Diskusi

"Infrastruktur yang dirancang dengan baik melahirkan performa bisnis yang andal."

Persiapkan materi kelompok Anda untuk Sidang Capstone Proyek di Pertemuan 14.